2026-7-22 Linux文件--磁盘与文件系统

- 1. 从进程中的文件到磁盘上的文件
  - 1.1 打开的文件与磁盘文件
  - 1.2 机械硬盘是什么
- 2. 机械硬盘的物理结构
  - 2.1 盘片、主轴、磁头和传动臂
  - 2.2 磁道、扇区、柱面与盘片
    - 2.2.1 磁道与扇区
    - 2.2.2 柱面
  - 2.3 容量与参数
- 3. CHS 寻址
  - 3.1 用 C、H、S 定位一个扇区
  - 3.2 早期 CHS 的容量限制
- 4. 从物理三维地址到线性 LBA
  - 4.1 把磁盘看作卷起来的磁带
  - 4.2 CHS 与 LBA 的对应关系
  - 4.3 谁负责 CHS 与 LBA 的转换
- 5. 从扇区到块
  - 5.1 为什么文件系统不直接逐扇区读写
  - 5.2 块号与 LBA
- 6. 分区、格式化与挂载
  - 6.1 分区
  - 6.2 格式化的本质
  - 6.3 实验：把普通文件当作一块小磁盘
  - 6.4 挂载
  - 6.5 怎样知道一个文件属于哪个分区
- 7. Ext 文件系统与块组
  - 7.1 Ext 系列
  - 7.2 分区与块组
  - 7.3 一个块组的构成
    - 7.3.1 超级块 Super Block
    - 7.3.2 块组描述符表 GDT
    - 7.3.3 块位图 Block Bitmap
    - 7.3.4 inode 位图 Inode Bitmap
    - 7.3.5 inode 表 Inode Table
    - 7.3.6 数据块 Data Blocks

- 8. inode：把文件属性和内容连接起来
  - 8.1 文件由属性与内容组成
  - 8.2 从 inode 到文件内容
  - 8.3 inode 号与块号的作用范围
  - 8.4 创建、删除、修改与查找
    - 8.4.1 创建文件
    - 8.4.2 删除文件
    - 8.4.3 修改文件
    - 8.4.4 查找文件
  - 8.5 目录也是文件
- 9. 路径解析与 dentry 缓存
  - 9.1 为什么访问文件必须有路径
  - 9.2 路径从哪里来
  - 9.3 dentry 与路径缓存
  - 9.4 打开文件时发生的主线
- 10. 硬链接与软链接
  - 10.1 硬链接
    - 10.1.1 硬链接的本质
    - 10.1.2 硬链接数
    - 10.1.3 .、.. 与目录的链接数
  - 10.2 软链接
    - 10.2.1 独立文件保存目标路径
    - 10.2.2 常见用途
  - 10.3 软链接与硬链接对比
- 11. 全篇主线回顾

1. 从进程中的文件到磁盘上的文件

1.1 打开的文件与磁盘文件

从进程视角看，文件首先表现为文件描述符 `fd`。进程通过 `read()`、`write()` 等接口读写文件时，操作系统接收到的核心参数通常是：

```
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
```

`buf` 的类型是 `void *`，操作系统并不根据数据内容是字符串、图片还是其他对象来决定底层读写方式。怎样解释这段数据，主要由用户程序决定。

- 文件已经打开时，进程通过文件描述符访问它。
- 文件没有打开时，内容长期保存在磁盘上的文件系统里。
- 文件操作优先使用语言层封装好的函数；只有需要特殊能力时，才直接使用系统调用。

因此，从磁盘文件系统的角度看：

> 文件 = 内容 + 属性

两者最终都要占用磁盘上的存储空间。本篇沿着“磁盘怎样定位一个存储单元 → 文件系统怎样组织一个分区 → inode 怎样把属性和内容连接起来”的顺序展开。缓冲区与 ELF 暂不展开。

1.2 机械硬盘是什么

课堂讨论的机械硬盘是一种具有明显机械结构的外设。它的速度比内存慢，但容量大、单位容量价格低。
向磁盘写数据的本质，是让磁头改变盘片表面磁性材料的磁化方向，用不同的磁化状态表示二进制的 `0` 和 `1`。现代固态硬盘没有这些机械运动部件，但“持久化存储、按块访问、由文件系统组织数据”的基本问题仍然存在。

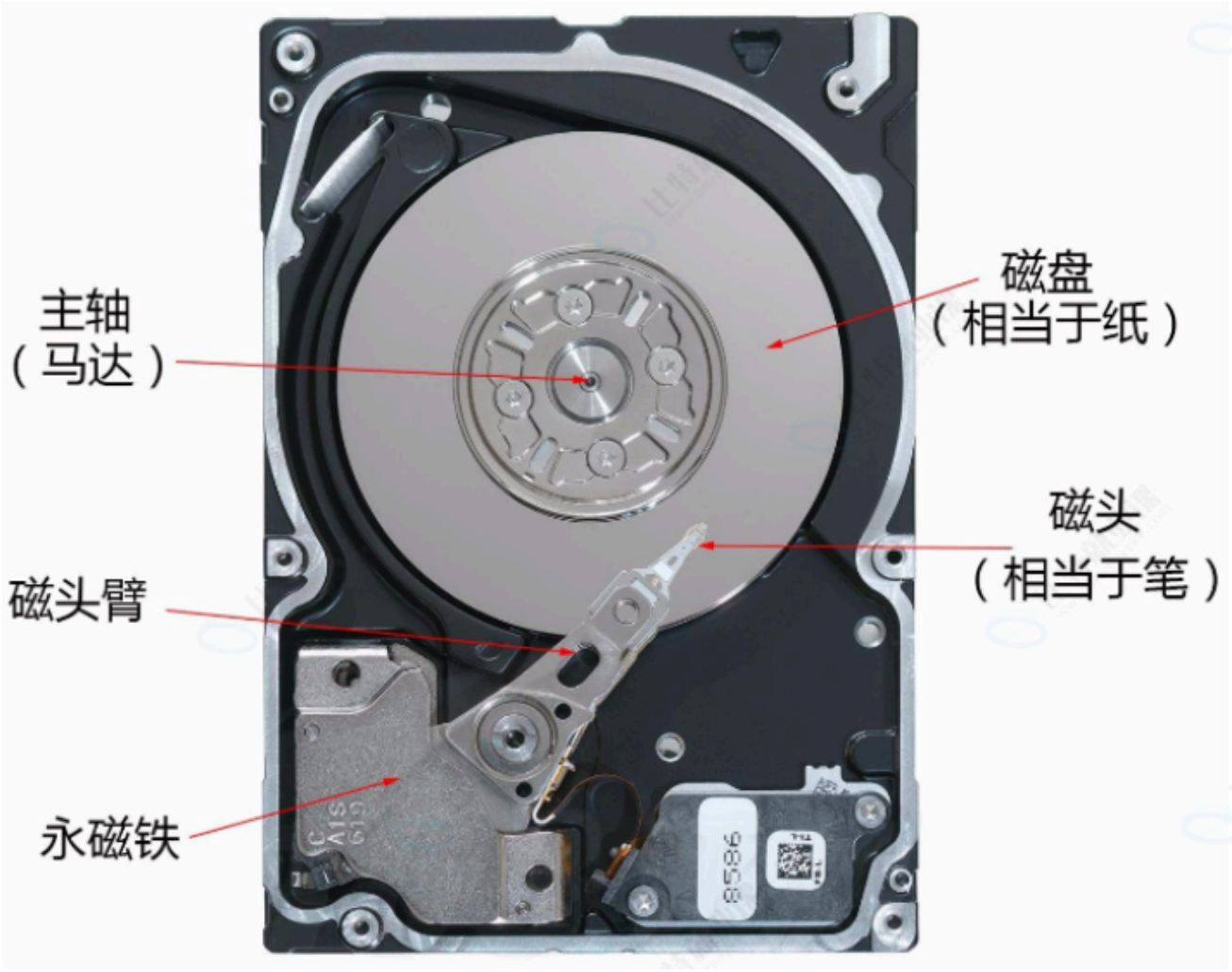
2. 机械硬盘的物理结构

2.1 盘片、主轴、磁头和传动臂

一个机械硬盘通常包含多张圆形盘片。盘片固定在主轴上一起旋转，每个盘片一般有上下两个盘面，每个盘面对应一个读写磁头。

> 所有磁头共进退，所有盘片共旋转。

磁头安装在传动臂上。所有磁头位于同一根传动臂组件上，因此它们会一起移动到相同半径的位置；盘片旋转时，磁头在对应盘面上的某一圈磁道上方经过。

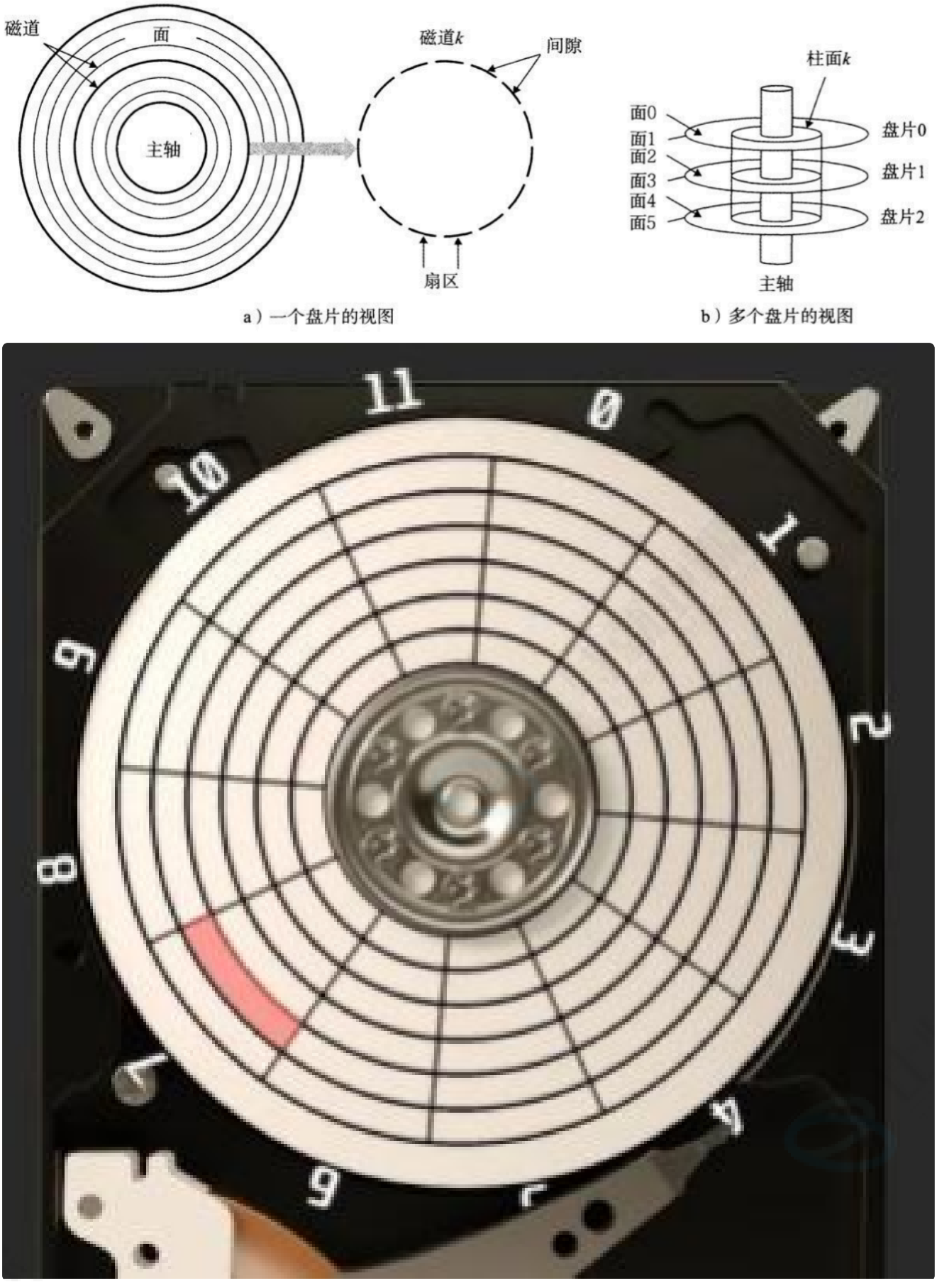


2.2 磁道、扇区、柱面与盘片

2.2.1 磁道与扇区

- 磁道：盘面上的同心圆。磁道一般从外圈向内圈编号为第 `0`、`1`、`2` 道。
- 扇区：把一条磁道分成若干段后得到的存储单元。
- 扇区是磁盘读写的传统最小单位，常见大小为 `512 B`。
- 课程为了简化计算，认为每条磁道的扇区数和每个扇区的容量都相同。

靠近主轴的一圈通常用于停放磁头，不用于保存普通数据。

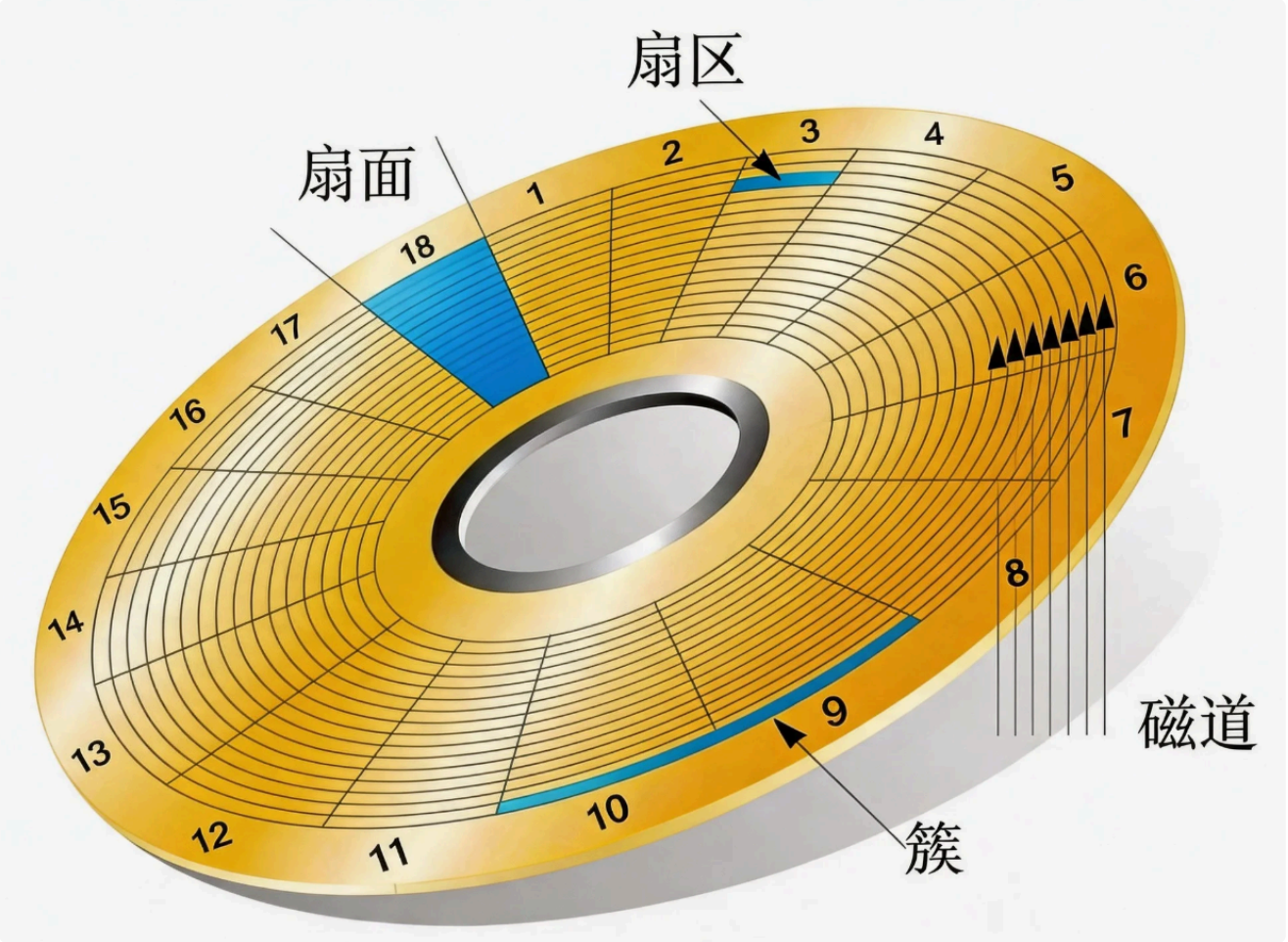


a）一个盘片的视图　　b）多个盘片的视图

2.2.2 柱面

多个盘面上半径相同的磁道，在逻辑上共同组成一个柱面。所有磁头共进退，因此磁头移动到一个柱面后，不必再次沿半径方向移动，就可以访问多个盘面上的同一半径位置。

磁头摆动的意义是选择某条磁道，也就是选择某个柱面；盘片旋转的意义是让目标扇区转到磁头下方。



2.3 容量与参数

机械硬盘的主要几何参数包括：

- 磁头数 `head`：通常等于有效盘面数。
- 磁道数、柱面数 `cylinder`：在课程模型中可以理解为每个盘面的磁道数量。
- 每道扇区数 `sector`：一条磁道中的扇区数量。
- 盘片数 `platter`：硬盘中盘片的数量。

磁盘容量可以写成：

```
磁盘容量 =
磁头数 × 柱面数 × 每道扇区数 × 每扇区字节数
```

在 Linux 中，可以用 `fdisk` 查看磁盘和分区信息：

```
sudo fdisk -l
```

3. CHS 寻址

3.1 用 C、H、S 定位一个扇区

CHS 是 `Cylinder-Head-Sector` 的缩写。要定位一个扇区，需要依次确定：

1. `Cylinder`：访问哪一个柱面，也就是哪一条同半径磁道。
2. `Head`：选择哪一个盘面，也就是哪一个磁头。
3. `Sector`：在该磁道中选择第几个扇区。

只要能定位一个扇区，就能用相似方式定位任意多个扇区。文件的内容和属性最终都可以落实到一组磁盘扇区上。

3.2 早期 CHS 的容量限制

早期系统常用：

- `8 bit` 表示磁头编号；
- `10 bit` 表示柱面编号；
- `6 bit` 表示扇区编号；
- 每个扇区 `512 B`。

因此课堂给出的最大容量为：

```
256 × 1024 × 63 × 512 B = 8064 MB
```

若按 `1 MB = 1000000 B` 计算，约为 `8.4 GB`。这个容量限制说明：直接暴露物理几何结构给上层使用并不合适。

4. 从物理三维地址到线性 LBA

4.1 把磁盘看作卷起来的磁带

磁带中保存数据时，相当于沿一条长带顺序排列。把磁带拉直，就得到一条线性存储空间。

机械磁盘虽然由盘片和同心圆扇区构成，但在逻辑上也可以把全部扇区依次排列，为每个扇区赋予一个连续编号。这个线性编号就是 LBA，即 Logical Block Address。



4.2 CHS 与 LBA 的对应关系

设：

- 柱面号为 `C`；
- 磁头号为 `H`；
- 扇区号为 `S`；
- 每磁道扇区数为 `SPT`；
- 磁头数为 `Heads`。

课程使用的换算公式为：

```
单个柱面的扇区数 = Heads × SPT

LBA = C × Heads × SPT + H × SPT + (S − 1)
```

这里减 `1` 是因为：

- `C`、`H` 从 `0` 开始编号；
- 传统 `S` 从 `1` 开始编号；
- `LBA` 从 `0` 开始编号。

反过来，由 LBA 求 CHS：

```
C = LBA // (Heads × SPT)
H = (LBA % (Heads × SPT)) // SPT
S = (LBA % SPT) + 1
```

其中 `//` 表示整除，`%` 表示取余。

这种换算本质上是把“柱面、磁头、扇区”这套三维下标压缩成一个线性下标。具体怎样改变一维下标的增长顺序，与磁盘展开时的排列顺序有关。

4.3 谁负责 CHS 与 LBA 的转换

上层操作系统可以只使用 LBA 访问磁盘：

```
操作系统 → LBA + 读写命令 → 磁盘控制器/固件 → CHS → 具体扇区
```

真实现代磁盘内部由控制器和固件处理逻辑地址与物理位置之间的映射。对于操作系统而言，磁盘可以被抽象成一个“元素为扇区的一维数组”，数组下标就是 LBA。

5. 从扇区到块

5.1 为什么文件系统不直接逐扇区读写

磁盘是典型的块设备。若每读写一次都要处理一个 `512 B` 的扇区，系统调用、地址计算和设备交互次数都会很多，效率较低。

因此，文件系统通常一次连续读写多个扇区，把它们组织成一个块。课程以最常见的 `4 KB` 块为例：

```
1 块 = 8 个连续扇区 = 8 × 512 B = 4096 B
```

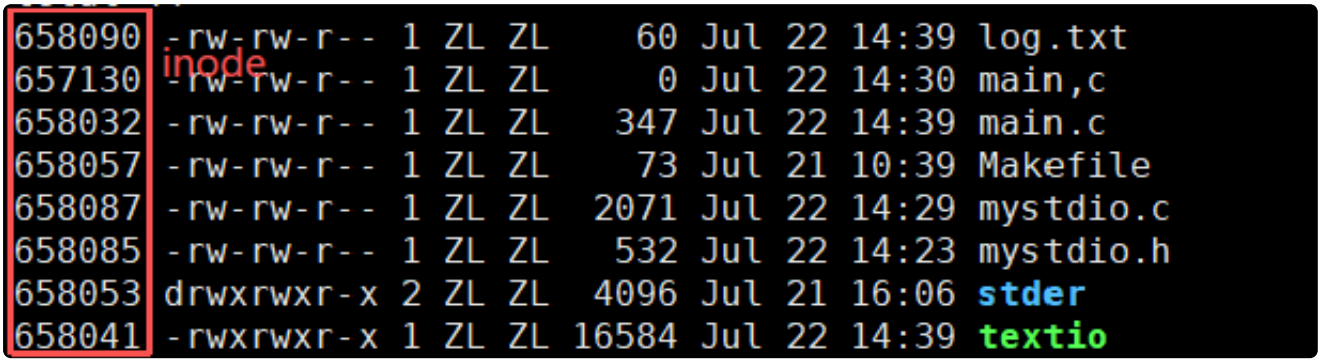
- 块大小通常在创建文件系统时确定。
- 要改变块大小，一般需要重新创建文件系统，不能只修改一个普通配置项。
- 块是文件系统分配文件内容的基本单位。

5.2 块号与 LBA

若一个块包含 8 个扇区：

```
起始 LBA = 块号 × 8
块内第 n 个扇区的 LBA = 块号 × 8 + n
块号 = LBA // 8
```

其中 `n` 的范围是 `0` 到 `7`。



需要注意：

- 磁盘物理上有多个面、磁道和扇区，可以看成三维结构。
- 对操作系统而言，可以先把全部扇区映射成线性 LBA。
- 文件系统再在 LBA 空间上划分块，并用块号组织文件内容。

6. 分区、格式化与挂载

6.1 分区

一块磁盘可以被分成多个分区。Windows 中的 `C:`、`D:` 等盘符通常对应不同分区；Linux 则把磁盘和分区作为 `/dev` 下的设备节点管理。

课堂用“柱面边界”解释分区：设置每个分区的起始柱面和结束柱面，再将分区内部的扇区映射为从 `0` 开始的 LBA 空间。

这是便于理解的教学模型。现代分区工具通常直接按扇区/LBA 边界划分，并常按 `1 MiB` 等边界对齐，不一定严格以柱面作为最小单位。

分区的意义在于先做“分治”：

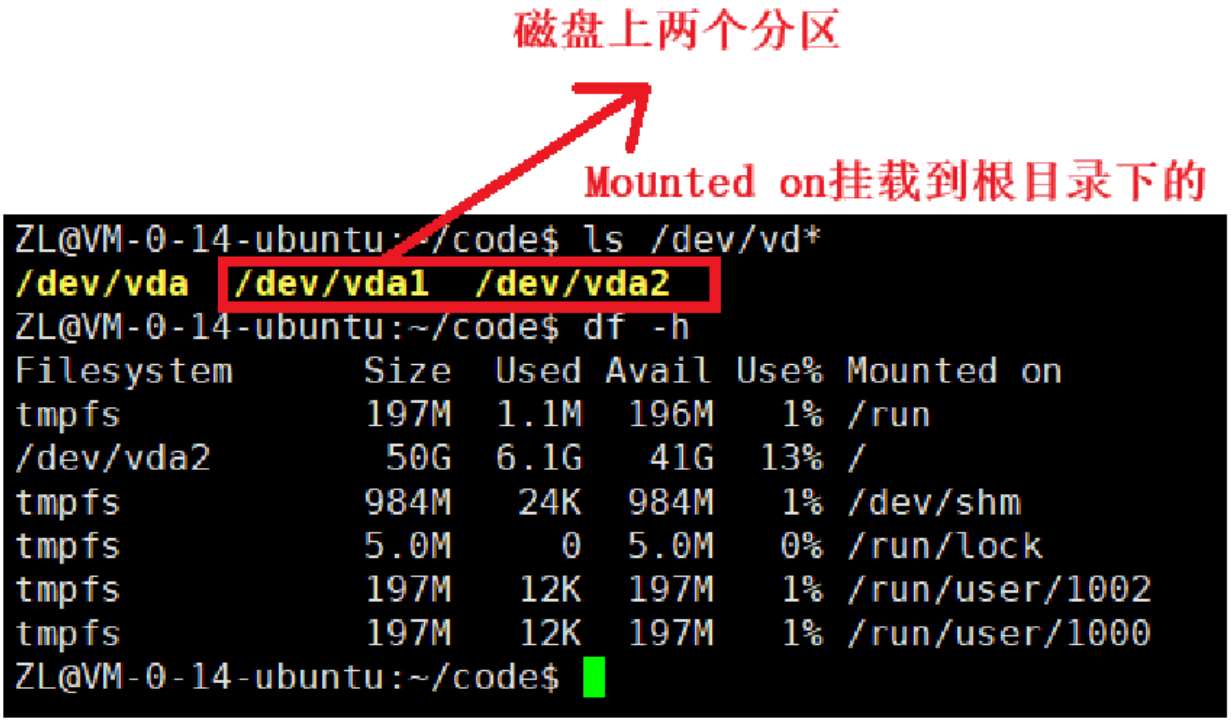
```
一整块磁盘
    ↓
若干分区
    ↓
每个分区建立自己的文件系统
```

6.2 格式化的本质

格式化并不是简单地“把数据全部写成 0”，其核心是向分区写入文件系统所需的元数据和管理结构，例如：

- 超级块；
- 块组描述符；
- 块位图；
- inode 位图；
- inode 表；
- 根目录等基础对象。

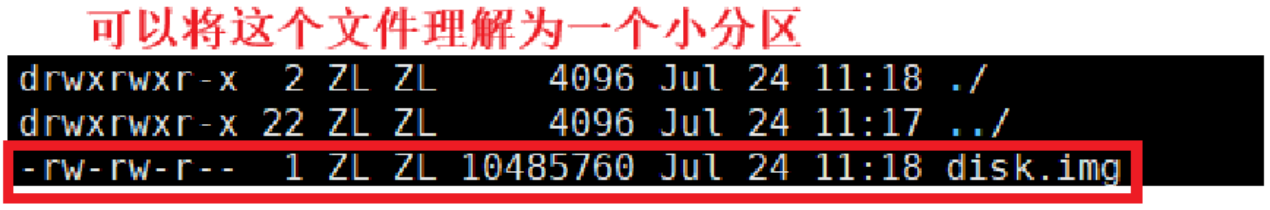
这些管理信息共同构成磁盘级文件系统。文件系统负责组织和管理分区中的数据块及文件属性。



6.3 实验：把普通文件当作一块小磁盘

先创建一个 `10 MB` 的普通文件，把它当作一个很小的“磁盘分区”：

```
dd if=/dev/zero of=./disk.img bs=1M count=10
```



把这个文件格式化为 Ext4 文件系统：

```
mkfs.ext4 ./disk.img
```

```
ZL@VM-0-14-ubuntu:~/code/lesson18$ mkfs.ext4 disk.img
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 2560 4k blocks and 2560 inodes

Allocating group tables: done
Writing inode tables: done
Creating journal (1024 blocks): done
Writing superblocks and filesystem accounting information: done
```

6.4 挂载

一个分区即使已经有文件系统，也不能直接拿路径来使用；需要先把它挂载到某个目录，建立“目录路径 → 文件系统根节点”的连接。

创建挂载点：

```
sudo mkdir -p /mnt/myvda
```

把 `disk.img` 挂载到该目录：

```
sudo mount -t ext4 ./disk.img /mnt/myvda
```

```
ZL@VM-0-14-ubuntu:~/code/lesson18$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           197M  1.1M  196M   1% /run
/dev/vda2        50G  6.1G   41G  13% /
tmpfs           984M   24K  984M   1% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           197M   12K  197M   1% /run/user/1002
tmpfs           197M   12K  197M   1% /run/user/1000
/dev/loop0      5.4M   24K  4.7M   1% /mnt/myvda
```

此后访问 `/mnt/myvda/text.c`，就会沿着该路径进入 `disk.img` 中的文件系统：

```
ZL@VM-0-14-ubuntu:/mnt/myvda$ ll
total 28
drwxr-xrwx 4 root root  4096 Jul 24 12:05 ./
drwxr-xr-x 3 root root  4096 Jul 24 11:38 ../
-rw-r--r-- 1 root root     0 Jul 24 12:03 hello.c
drwx------ 2 root root 16384 Jul 24 11:28 lost+found/
drwxr-xr-x 2 root root  4096 Jul 24 12:02 text.c/
```

卸载时执行：

```
sudo umount /mnt/myvda
```

```
ZL@VM-0-14-ubuntu:~/code$ sudo umount /mnt/myvda
ZL@VM-0-14-ubuntu:~/code$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           197M  1.1M  196M   1% /run
/dev/vda2        50G  6.1G   41G  13% /
tmpfs           984M   24K  984M   1% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
tmpfs           197M   12K  197M   1% /run/user/1002
tmpfs           197M   12K  197M   1% /run/user/1000
```

挂载普通镜像文件时，Linux 常通过 loop 设备把该文件暴露为块设备，因此 `df -h` 中可能看到 `/dev/loop0`。卸载后文件系统本身仍然保存在 `disk.img` 中，只是暂时不再从这个目录可见。

6.5 怎样知道一个文件属于哪个分区

文件必须有路径。路径前缀决定了访问首先进入哪一个挂载点，因此也决定了进入哪一个分区和文件系统。例如：

```
/mnt/myvda/text.c
```

其中 `/mnt/myvda` 是挂载点，后续的 inode 和块号都在挂载到该位置的 `disk.img` 文件系统中解释。

7. Ext 文件系统与块组

7.1 Ext 系列

Ext2 是早期 Linux 常用文件系统。Ext3 增加了日志机制，Ext4 又进一步增加扩展能力，但它们仍保留了以块组、inode 和数据块为核心的组织思路。

课程重点不是各个版本的完整功能差异，而是通过 Ext2 理解：

> 文件系统怎样管理属性、内容、空闲 inode 和空闲数据块。

7.2 分区与块组

Ext2 文件系统把一个分区划分为若干个大小相同的块组 `Block Group`。分区前面还有启动块等信息，之后才是普通文件系统区域。

课程模型中的整体层次可以概括为：

```
磁盘
  └ 分区
      └ 文件系统
          └ 若干块组
              └ 管理信息 + inode + 数据块
```

7.3 一个块组的构成

Ext2 块组的主要结构包括：

```
Super Block
GDT
Block Bitmap
Inode Bitmap
Inode Table
Data Blocks
```

7.3.1 超级块 Super Block

超级块描述整个文件系统的整体信息，例如：

- 总 inode 数和总块数；
- 空闲 inode 数和空闲块数；
- 块大小、inode 大小；

- 每块组中的块数、inode 数；
- 挂载与写入等管理时间。

超级块非常重要。若主超级块损坏，文件系统结构可能难以解释，因此 Ext 文件系统会在部分块组中保存备份。

7.3.2 块组描述符表 GDT

GDT，即 `Group Descriptor Table`，描述各块组的属性。一个块组对应一个块组描述符，记录例如：

- 块位图在哪个块；
- inode 位图在哪个块；
- inode 表从哪个块开始；
- 本组空闲块数、空闲 inode 数；
- 本组目录数。

7.3.3 块位图 Block Bitmap

块位图中的每一个 bit 对应数据区中的一个块：

- `1` 表示该块已被占用；
- `0` 表示该块空闲。

创建文件需要分配数据块时，文件系统会查找块位图中的空闲位。

7.3.4 inode 位图 Inode Bitmap

inode 位图中的每一个 bit 对应一个 inode：

- `1` 表示该 inode 已分配；
- `0` 表示该 inode 空闲。

创建文件需要新建 inode 时，文件系统从 inode 位图中分配一项。

7.3.5 inode 表 Inode Table

inode 表存放当前块组中全部文件属性记录。课堂使用的典型大小是：

```
一个 inode ≈ 128 B
一个 4 KB 块可放 32 个 inode
```

Ext2 inode 既可表现为 `128 B`，也可根据文件系统配置使用更大尺寸；课堂统一按 `128 B` 理解。

7.3.6 数据块 Data Blocks

数据块区存放文件内容，也存放目录文件内容。目录文件的数据块中保存的是：

```
目录项文件名 → 对应 inode 号
```

文件名本身不属于 inode 的普通属性字段。一个目录项把“名字”和“inode 号”连接起来，而 inode 再通过块指针连接文件内容。

8. inode：把文件属性和内容连接起来

8.1 文件由属性与内容组成

Linux 中文件的属性和内容分开存储：

- 属性：保存在 inode 中；
- 内容：保存在数据块中。

可通过 `ls -l` 查看常见属性，包括模式、硬链接数、所有者、用户组、大小、修改时间和文件名；`stat` 还能显示 inode 号、设备号、块数及多个时间字段。

Ext2 inode 中和本篇关系最密切的字段包括：

```
i_mode          文件类型与权限
i_uid / i_gid   所有者与用户组
i_size          文件大小
i_links_count   硬链接数
i_blocks        占用的 512 B 计数块数
i_block[15]     数据块指针
```

文件名不属于 inode 内部的普通属性字段。目录文件通过目录项保存“文件名 → inode 号”的映射。

8.2 从 inode 到文件内容

Ext2 inode 提供 `15` 个块指针位置：

```
i_block[0]  ~ i_block[11]  12 个直接块指针
i_block[12]                1 个一级间接块指针
i_block[13]                1 个二级间接块指针
i_block[14]                1 个三级间接块指针
```

- 小文件可以直接由前 12 个指针找到数据块。
- 大文件可通过间接块保存更多数据块号。
- 间接块本身也占一个块，块中存放的是一组“块号”。

因此，只要给定分区和 inode 号，就能沿着：

```
inode 号
  → inode 表项
  → inode 属性
  → i_block[]
  → 数据块
```

找到文件的属性与内容。

8.3 inode 号与块号的作用范围

- inode 号在整个文件系统分区内具有意义，可以跨越块组定位，但不能直接跨分区。
- 数据块号也在所属分区内解释。
- 不同分区可以各自从相似的 inode 号或块号开始，因此读取它们时必须先确定分区和文件系统。

8.4 创建、删除、修改与查找

8.4.1 创建文件

创建一个新文件大致需要：

1. 在 inode 位图中找到一个空闲 inode。
2. 把文件属性写入对应 inode 表项。
3. 在块位图中分配保存内容所需的数据块。
4. 把数据写入数据块。
5. 在 inode 的块指针中记录数据块位置。
6. 在当前目录文件里新增“文件名 → inode 号”这一目录项。

课程示例中，新建的 inode 号为 `263466`，文件内容被分配到 `300`、`500`、`800` 等数据块，最后再把目录项 `(abc, 263466)` 写入当前目录。

8.4.2 删除文件

删除一个文件通常是：

1. 在当前目录中删除“文件名 → inode 号”的目录项。
2. 将 inode 的硬链接数减 `1`。
3. 当硬链接数变成 `0` 时，释放该 inode 和数据块。

因此，删除某个文件名并不等于立刻物理擦除全部内容。普通删除主要修改目录项、链接计数和位图状态，所以数据可能在一段时间内仍有恢复机会。如果对应 inode 或数据块随后被其他文件复用，恢复就会变得困难。

8.4.3 修改文件

修改文件涉及内容或属性变化。内核先根据 inode 号找到属性和数据块位置，再把需要修改的数据块读入内存，更新后按文件系统策略写回磁盘。

若只是修改少量内容，不一定要一次性遍历整个文件；但文件系统必须维护文件长度、块指针、占用计数和属性时间等一致信息。

8.4.4 查找文件

查找文件时，不能只看文件名本身。要从已经确定的目录文件出发：

1. 在目录的数据块中找到文件名；
2. 由目录项得到 inode 号；

3. 根据 inode 号找到 inode；
4. 通过 `i_block[]` 找到数据块；
5. 读取内容或展示属性。

8.5 目录也是文件

在磁盘和文件系统看来，目录与普通文件都使用同样的基本模型：

```
属性（inode） + 内容（数据块）
```

区别体现在数据块保存的数据上：

- 普通文件的数据块保存用户内容；
- 目录的数据块保存目录项，即文件名与 inode 号的映射。

这也解释了目录权限的意义：

- 没有读权限时，难以列出目录中的数据块内容；
- 没有写权限时，不能向目录文件的数据块中增加、删除或修改目录项；
- 没有执行/搜索权限时，即使知道子文件名，也不能正常穿过该目录解析路径。

```
ZL@VM-0-14-ubuntu:~/code$ ls -lid lesson17
658052 drwxrwxr-x 2 ZL ZL 4096 Jul 23 09:00 lesson17
ZL@VM-0-14-ubuntu:~/code$
```

9. 路径解析与 dentry 缓存

9.1 为什么访问文件必须有路径

若想访问：

```
/home/whb/code/test/test.c
```

就必须从根目录 `/` 开始，依次解析 `home`、`whb`、`code`、`test`，最后在 `test` 目录的数据块中找到 `test.c` 对应的 inode 号。

这看起来像递归：要打开当前目录，先要打开它的父目录；继续向上，最终必须有一个固定且明确的入口。这个入口就是根目录 `/`。

因此：

```
目录名 + 文件名 → 路径
```

路径解析把用户给出的路径与具体挂载点、目录项和 inode 逐步连接起来。

9.2 路径从哪里来

进程发起文件访问时，会提供路径。进程还有当前工作目录 `cwd`。相对路径需要先以 `cwd` 为参照点，绝对路径则直接从 `/` 开始。

整个系统的路径结构由根目录、系统默认目录、用户目录以及用户之后创建的目录共同形成。每个新建文件都位于某个目录之下，因此天然有路径可以解析。

9.3 dentry 与路径缓存

如果每次访问都从 `/` 走一遍会太慢。Linux 内核用 `struct dentry` 在内存中维护路径结构：

- dentry 把一个目录项的名称关联到对应 inode；
- 多个 dentry 通过父子关系组成树；
- 目录和普通文件都可以有 dentry；
- 打开的路径会进入缓存，便于随后快速查找；
- 缓存节点还会参与哈希查找和 LRU 淘汰。

第一次沿某条路径访问时可能需要从磁盘加载；路径已经被缓存后，再次访问通常会更快。

9.4 打开文件时发生的主线

一次 `open()` 的核心主线可以概括为：

```
进程提供路径
  → 确认当前目录/挂载点
  → 在 dentry 树中解析路径
  → 找到目标 dentry
```

```
→ 加载或复用 inode
→ 创建打开文件对象并返回 fd
```

缓冲区如何参与页缓存和写回，将在后续内容中展开。本篇只到“路径解析定位 inode，inode 再定位数据块”为止。

10. 硬链接与软链接

10.1 硬链接

10.1.1 硬链接的本质

硬链接让多个文件名映射到同一个 inode：

```
ln log.txt log_hard
```

执行后，`log.txt` 与 `log_hard` 的 inode 号相同。它们不是两个拥有独立内容的文件，而是同一文件的两个目录项。

```
ZL@VM-0-14-ubuntu:~/code/lesson18$ echo "hhhhhhh" >> log.txt
ZL@VM-0-14-ubuntu:~/code/lesson18$ ln log.txt log_hard
ZL@VM-0-14-ubuntu:~/code/lesson18$ ll
```

10.1.2 硬链接数

inode 中的 `i_links_count` 记录有多少个目录项指向它：

```
658099 -rw-rw-r-- 2 ZL ZL        8 Jul 24 12:44 log_hard
658099 -rw-rw-r-- 2 ZL ZL        8 Jul 24 12:44 log.txt
```

删除一个硬链接时：

```
删除目录项 + inode 硬链接数减 1
```

只有硬链接数变为 `0`，对应 inode 和数据块才会成为可回收空间。新建普通文件通常只有一个目录项，所以硬链接数为 `1`。

硬链接能避免为同一个大文件复制多份数据，但它不是独立的数据备份：多个名字仍然指向同一份内容。若原文件内容被修改，所有硬链接看到的都是修改后的内容。

10.1.3 `.`、`..` 与目录的链接数

新建目录时，内核会创建至少两个特殊目录项：

- `.`：指向当前目录自身；
- `..`：指向父目录。

因此，一个新目录的基础硬链接数是 `2`。每当父目录下增加一个子目录，新子目录中的 `..` 会指向父目录，因此父目录的硬链接数再增加 `1`。

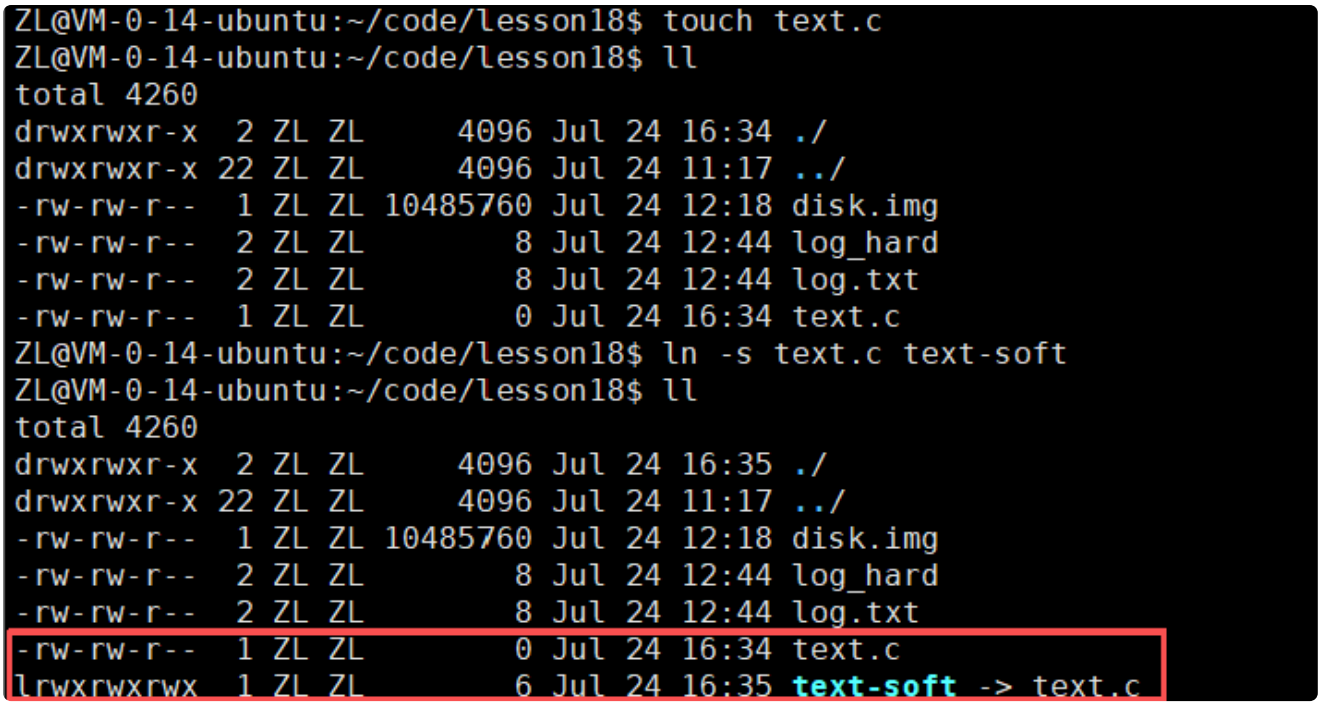
普通用户创建的硬链接通常不能直接指向目录，因为目录中的 `..` 和子树关系容易形成环，并可能破坏路径树的一致性。操作系统在构建目录层次时使用相关链接机制，和用户执行 `ln` 的限制不是同一层面的事情。

10.2 软链接

10.2.1 独立文件保存目标路径

软链接命令：

```
ln -s text.c text-soft
```



软链接是一个独立文件，有自己的 inode。它的数据内容是目标文件路径字符串，例如 `text.c`。

软链接与目标文件的关系是：

```
软链接目录项 → 软链接自己的 inode → 数据块保存目标路径
                                          ↓
                                    重新解析目标路径
```

若目标被删除或改名，软链接可能变成悬空链接。

10.2.2 常见用途

软链接类似快捷方式，可以用于：

- 让不同路径访问同一程序或库；
- 保留稳定的命令名，同时指向具体版本；
- 把软件放入 `/usr/bin` 等命令目录而不复制完整文件。

```
ZL@VM-0-14-ubuntu:~/code/lesson18$ ln -s ./bin/dir1/dir2/dir3/myexe run
ZL@VM-0-14-ubuntu:~/code/lesson18$ ll
total 4276
drwxrwxr-x  5 ZL ZL     4096 Jul 24 20:12 ./
drwxrwxr-x 22 ZL ZL     4096 Jul 24 11:17 ../
drwxrwxr-x  3 ZL ZL     4096 Jul 24 20:10 bin/
drwxrwxr-x  2 ZL ZL     4096 Jul 24 20:01 conf/
-rw-rw-r--  1 ZL ZL 10485760 Jul 24 12:18 disk.img
drwxrwxr-x  2 ZL ZL     4096 Jul 24 20:03 log/
-rw-rw-r--  2 ZL ZL        8 Jul 24 12:44 log_hard
-rw-rw-r--  2 ZL ZL        8 Jul 24 12:44 log.txt
lrwxrwxrwx  1 ZL ZL       26 Jul 24 20:12 run -> ./bin/dir1/dir2/dir3/myexe*
-rw-rw-r--  1 ZL ZL        5 Jul 24 20:00 text.c
lrwxrwxrwx  1 ZL ZL        6 Jul 24 16:35 text-soft -> text.c
ZL@VM-0-14-ubuntu:~/code/lesson18$ ./
-bash: ./: Is a directory
ZL@VM-0-14-ubuntu:~/code/lesson18$ ./run
hello world!
ZL@VM-0-14-ubuntu:~/code/lesson18$ tree
```

```
ZL@VM-0-14-ubuntu:~/code/lesson18$ sudo ln -s /home/ZL/code/lesson18/bin/dir1/dir2/dir3/myexe
 /usr/bin/myexe
ZL@VM-0-14-ubuntu:~/code/lesson18$ pwd
/home/ZL/code/lesson18
ZL@VM-0-14-ubuntu:~/code/lesson18$ sudo ln -s /home/ZL/code/lesson18/bin/dir1/dir2/dir3/myexe
 /usr/bin/myexe
ln: failed to create symbolic link '/usr/bin/myexe': File exists
ZL@VM-0-14-ubuntu:~/code/lesson18$ ls /usr/bin/myexe
/usr/bin/myexe
ZL@VM-0-14-ubuntu:~/code/lesson18$ ls /usr/bin/myexe -l
lrwxrwxrwx 1 root root 47 Jul 24 20:25 /usr/bin/myexe -> /home/ZL/code/lesson18/bin/dir1/dir2
/dir3/myexe
ZL@VM-0-14-ubuntu:~/code/lesson18$ myexe
hello world!
ZL@VM-0-14-ubuntu:~/code/lesson18$
```

若不再需要这个软链接，可以只删除链接本身：

```
sudo unlink /usr/bin/myexe
```

10.3 软链接与硬链接对比

对比项	硬链接	软链接
是否有独立 inode	否	是
与原文件的 inode 号	相同	不同
保存的数据	共同指向同一份文件内容	保存目标路径字符串
删除原文件名	其他硬链接仍可访问内容	可能失效
能否跨文件系统	通常不能	可以
典型用途	多名称引用同一文件、`./..`	快捷方式、命令与版本切换

11. 全篇主线回顾

将本篇内容串联起来，可以得到一条完整的访问链：

```
机械磁盘
  → CHS 物理定位
  → LBA 线性扇区地址
  → 多个扇区组成块
  → 分区与格式化
  → Ext2 块组
  → inode 保存属性与块指针
  → 数据块保存文件内容
  → 目录块保存“文件名 → inode 号”
  → 路径解析与 dentry 缓存
  → 进程通过 fd 访问已打开文件
```

磁盘负责保存二进制数据，文件系统负责给这些空间建立可管理的结构，inode 负责把“文件是什么”和“文件内容在哪里”连接起来，目录与路径则负责让用户通过文件名稳定地找到对应 inode。